

Agentic AI Gateway

One API in front of every LLM provider: reliable, cost-controlled, safe, and observable

Project presentation report · September 2026

The problem

Agentic applications depend on several LLM providers at once, and each one differs:

- **Different APIs:** OpenAI, Anthropic, Gemini, and Ollama each have their own request format, streaming format, and tool-calling format
- **Different failures:** rate limits, timeouts, and outages happen at different times on different providers
- **Different prices:** the same task can cost 16x more on one model than another
- **Agent loops multiply traffic:** RAG, tool calls, and MCP turn one user question into many model calls, so cost and failure risk multiply with them

Today each application solves this on its own, if at all.

The solution

A **gateway** between applications and providers:

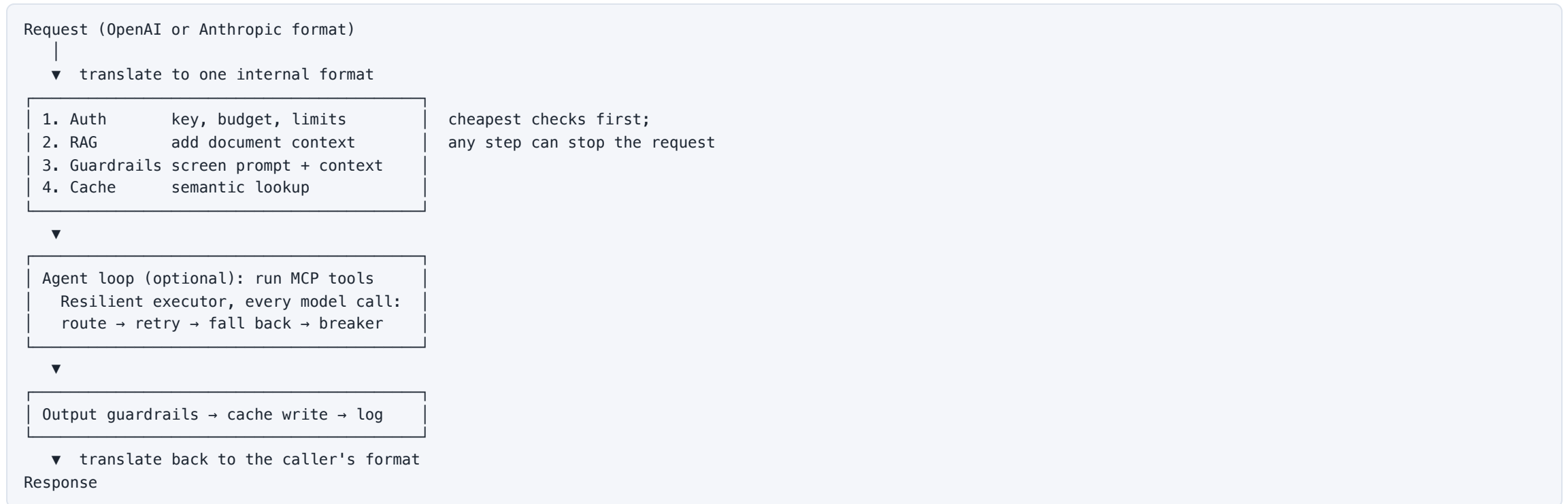


- Applications speak **one API**; the existing OpenAI and Anthropic SDKs work unchanged
- The gateway handles **routing, retries, fallback, caching, budgets, guardrails, RAG, tools, and logging**
- Operators get a **web console** to manage keys, models, costs, and safety

What it does

Capability	What it gives you
Unified API	OpenAI- and Anthropic-compatible endpoints; any client reaches any provider
4 native providers	OpenAI, Anthropic, Gemini, Ollama, written directly against each API (no LiteLLM)
Reliability	Retries with backoff, cross-provider fallback, circuit breakers
Smart routing	Priority, least-cost, lowest-latency, weighted, conditional
Semantic cache	Answers near-identical questions from Redis vector search
Cost control	Virtual keys and teams with budgets, rate limits, allowlists
Guardrails	PII, regex, denylist, and LLM-judge rules on input and output, including streams
RAG and MCP	Ground answers in documents; let the gateway run tools for the model
Observability	Prometheus, OpenTelemetry, structured logs, per-request history
Console	React web app for operators, with admin and viewer roles

How a request flows



Design choices: unauthenticated work costs nothing; blocked prompts never touch the cache; the log records exactly what the client received.

Reliability by design

- **Retry** the same deployment for transient errors (rate limit, timeout, 5xx), with randomised backoff so replicas don't retry in lockstep
- **Fall back** to another deployment, even on another provider, for errors such as "context too long"
- **Circuit breaker** skips a failing deployment, then probes it with one request after a cool-down
- **Streams fail over only before the first token**, so two models' answers are never spliced together
- **An unknown model is rejected** (404), never silently served by a different, possibly pricier, model
- **Degraded modes**: if the cache or guardrails fail, traffic keeps flowing; if auth fails, the gateway refuses to start

Evaluation method

Every figure comes from `scripts/evaluate.py` and `scripts/loadtest.py` driving a **running gateway over HTTP**: the real app, real Postgres, and real Redis Stack.

- A **deterministic fake upstream** stands in for the providers: free, offline, repeatable, and able to fail on demand
- It speaks all four providers' native formats, so every adapter's translation code runs
- An **evaluation catalogue** makes behaviour measurable:
 - `eval-chat` : primary deployment is deliberately dead, which forces real failover
 - `eval-router` : two deployments with different prices and weights, so routing strategies must disagree
 - `eval-multi` : dead Anthropic deployment that falls back to Gemini

Latency figures measure the gateway's own overhead, not model time.

Results: the grading criteria

Criterion	Result
Request success rate	100% (50 of 50), including a forced failover
Latency (gateway overhead)	p50 15 ms · p95 20 ms · p99 22 ms
Time to first token (stream)	p50 5 ms
Provider failover	✅ recovered from a dead primary; served by the fallback
Cache hit ratio	100% on repeated prompts; 0 false hits on an unrelated prompt
Cost per successful request	\$0.000191 (1,440 tokens, \$0.0095 total)
Routing effectiveness	✅ strategies diverge: least-cost → cheap deployment, priority → preferred one
Guardrails	✅ benign prompt allowed; PII blocked or redacted
SDK compatibility	10/10 official OpenAI and Anthropic SDK checks

Results: sustained load

20 seconds of continuous traffic at concurrency 15:

Metric	Value
Requests	4,668
Failed	0
Throughput	232.8 requests/s
p50 / p95 / p99	66.9 ms / 98.6 ms / 114.1 ms
Latency drift (start → end)	-6.1 ms: no build-up, no saturation
Cache hit ratio	30.9% against a configured 30% repeatable share

The measured cache hit ratio matching the configured share shows the cache hits exactly what it should, across thousands of mixed concurrent requests.

Engineering quality: test-driven

Four automated suites, two of them enforced as **coverage gates in CI**:

Suite	Scope	Result
Unit	Every module in isolation	746 tests
Integration	Real app on real Postgres + Redis Stack	208 tests
Backend total	Unit + integration combined	954 passing · 100% line and branch coverage (gate)
Console unit	API client, streaming parser, auth, components	68 passing · 100% coverage of logic (gate)
End-to-end	Cucumber: official SDKs + real browser against the dockerised stack	121 scenarios passing

Plus lint (ruff), type checking (mypy, tsc), a Helm chart check, and a Docker build on every push.

What testing caught

Testing against the **assembled system** found defects that mocked tests missed. All are fixed, each with a regression test:

- **Semantic cache never hit:** Redis's RESP3 reply shape wasn't parsed; later, binary embeddings broke decoding. The only symptom was a silent 0% hit ratio
- **Budgets were never charged**, and streamed or failed requests were never logged
- **Security:** any virtual key could register a stdio MCP server, i.e. run commands on the host
- **Streamed provider errors crashed** instead of being reported, which also blocked context-length fallback
- **Anthropic streaming** was broken for the official TypeScript SDK
- **Console:** deep links 404'd; the first admin was never created on a fresh install; the playground never sent the message just typed

Lesson: mocks encode assumptions, and the assumptions were what was wrong. 26 defects were found and fixed through verification.

Feature completeness

128 features complete and tested · 5 partial · 5 proposed

Complete: every data-plane endpoint, all four provider adapters (now tested end to end), routing, resilience, cache, guardrails, cost control, RAG, MCP, the agent loop, observability, and all console pages.

Partial (implemented, not yet tested end to end):

- Playground RAG, compare, and replay-from-log modes
- Regenerating keys and editing limits from the console (API is tested)
- Docker Compose, Grafana, and Prometheus setup (checked manually)

Proposed: guardrails on tool traffic · streaming intermediate agent steps · PDF/DOCX ingestion · load test in CI · a live-provider test suite

Honest limitations

- **Paraphrase caching is not demonstrated.** The fake upstream's embeddings are hash-based, so the evaluation proves exact-repeat hits and correct rejection, not semantic matching. That needs a real embedding model
- **Latency excludes real provider time** by design: it isolates the gateway's cost
- **Provider adapters are tested against faithful fakes**, not live vendor APIs; a vendor changing its format would not be caught
- **Direct MCP tool calls** (`/v1/mcp/tools/call`) are not yet logged, costed, or guardrailed
- **Costs are estimates** from price tables, not invoice-exact

Deployment

- **Docker image:** multi-stage build with the console bundled; no Node runtime in production
- **Docker Compose:** gateway, Postgres, Redis Stack, Prometheus, Grafana, Ollama
- **Kubernetes:** Helm chart with a migration init container
- **Database migrations:** Alembic, via `aigateway migrate`
- **CLI:** `migrate`, `create-admin`, `create-key`, `routes`, `serve`

Setup, configuration, and operation are covered step by step in the **User Manual**.

Summary

- **One API, four providers:** existing SDKs and coding agents work unchanged
- **Reliable:** 100% success with a dead primary; 0 failures across 4,668 requests under load
- **Cheap to run through:** ~15 ms median overhead; every request's cost is known
- **Safe and controlled:** budgets, rate limits, allowlists, and guardrails on streams
- **Proven by tests:** 100% backend coverage, 954 + 68 unit/integration tests, 121 end-to-end scenarios

Next steps

Live-provider test suite · guardrails on tool traffic · PDF/DOCX ingestion · load test gate in CI

Thank you

Questions?

Source, documentation, and the feature-to-test map (`features.md`) are in the project repository.